

Dans le cadre de ce tp, l'objectif était de mettre en place un pare-feu sur une VM Linux avec différentes règles .

Le travail consistait à mettre en place une politique par défaut en sortie, permettre ICMP en entrée, interdire le HTTP en sortie, héberger une page accessible en http, bloquer les IP des voisins contigus sur le WAN/LAN et enfin bloquer l'accès à « leboncoin ».

Pour ce faire, il existe plusieurs outils que nous pouvons utiliser, l'outil choisi ici pour réaliser ce tp est iptables intégré directement à Linux.

SOMMAIRE :

1- Mise en place d'une politique par défaut (en sortie)

2- Autorisé le protocole ICMP en entrée

3- Interdire le HTTP en sortie

4- Héberger une page accessible en http

5- Bloquer les IP des voisins contigus

6- Bloquer Leboncoin

Pour commencer le TP, on va d'abord installer les outils nécessaires qui sont donc iptables (parefeu) et apache pour la page web avec ces deux commandes :

```
root@debian:~# sudo apt install iptables
Installation de :
  iptables

Installation de dépendances :
  libip4tc2 libip6tc2

Paquets suggérés :
  firewalld

Sommaire :
  Mise à niveau de : 0. Installation de : 3Supprimé : 0. Non mis à jour : 0
Taille du téléchargement : 401 kB
Espace nécessaire : 2 599 kB / 107 GB disponible

Continuer ? [O/n] o
```

```
root@debian:~# sudo apt install apache2 -y
```

Une fois les outils installer nous pouvons commencer par la mise en place d'une politique par défaut en sortie .

1- Mise en place d'une politique par défaut (en sortie)

Pour ce faire, il faut autoriser uniquement les sorties et refusé tout entrant avec ces commande :

```
root@debian:~# sudo iptables -P INPUT DROP
root@debian:~# sudo iptables -P FORWARD DROP
root@debian:~# sudo iptables -P OUTPUT ACCEPT
```

les deux premières commandes « INPUT » et « FORWARD » interdisent les entrées et la troisième commande « OUTPUT » permet d'accepter les sorties.

Ensuite, il faut ajouter une règle qui autorise tout le trafic qui vient de notre propre machine en localhost . Cela permet aux applications de notre propre machine de pouvoir toujours communiquer en local comme par exemple Apache . Si cette règle n'est pas ajoutée, certaines applications locales pourraient ne plus fonctionner . Voici la commande qu'il faut utiliser :

```
root@debian:~# sudo iptables -A INPUT -i lo -j ACCEPT
```

- « INPUT » correspond aux paquets entrants
- « -i lo » correspond à loopback (localhost)
- « ACCEPT » permet d'autoriser ces paquets

Ensuite on fait la même chose mais pour autoriser le trafic que notre machine génère et qui a pour sortir notre propre machine (localhost) . Cela permet à notre machine de pouvoir toujours effectuer des requêtes en local et recevoir des réponses .

```
root@debian:~# sudo iptables -A OUTPUT -o lo -j ACCEPT
```

Enfin, on fait cette commande qui permet d'autoriser les paquets entrants réponses à des connexions déjà ouvertes afin de permettre à Internet et aux services de fonctionner correctement

```
root@debian:~# sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED
-j ACCEPT
root@debian:~#
```

2- Autoriser le protocole ICMP en entrée

La politique par défaut avec sortie uniquement est mise en place, maintenant nous pouvons passer à l'autorisation d'ICMP en entrée pour que les autres machines puissent me ping .

Il suffit juste d'ajouter une règle avec cette commande :

```
root@debian:~# sudo iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
```

Pour vérifier que le ICMP fonctionne, on peut tester de ping notre machine à partir d'une autre machine et les paquets devraient bien arrivés comme ci dessous (l'ip de ma machine est 10.10.7.6) :

```
root@debian:~# ping 10.10.7.6
PING 10.10.7.6 (10.10.7.6) 56(84) bytes of data.
64 bytes from 10.10.7.6: icmp_seq=1 ttl=64 time=0.499 ms
64 bytes from 10.10.7.6: icmp_seq=2 ttl=64 time=0.640 ms
64 bytes from 10.10.7.6: icmp_seq=3 ttl=64 time=0.545 ms
^C
--- 10.10.7.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2034ms
rtt min/avg/max/mdev = 0.499/0.561/0.640/0.058 ms
root@debian:~#
```

Le ICMP en entrée fonctionne , nous pouvons donc passer à l'interdiction du HTTP en sortie .

3- Interdire le HTTP en sortie

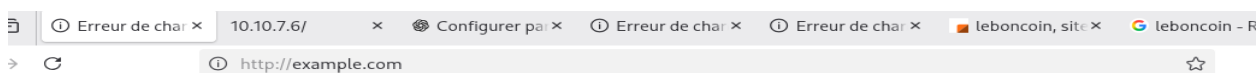
Pour bloquer le HTTP en sortie, c'est à dire ne pas pouvoir accéder par exemple aux sites qui sont en http, il faut ajouter une règle qui bloque le port 80. Le port 80 est le port dédié au protocole http, donc si on bloque l'écoute sur ce port, le http sera bloqué . Le https lui restera fonctionnel car il correspond à un autre port (443).

Nous allons donc ajouter une règle pour bloquer ce port en sortie avec cette commande :

```
root@debian:~# sudo iptables -A OUTPUT -p tcp --dport 80 -j REJECT
```

cette règle permet de bloquer tous les paquets en sortie du port 80 (http) .

Il faut maintenant tester si le blocage fonctionne, il suffit de se rendre sur un site en http par exemple « <http://example.com> » et on peut voir que la connexion est bloqué :



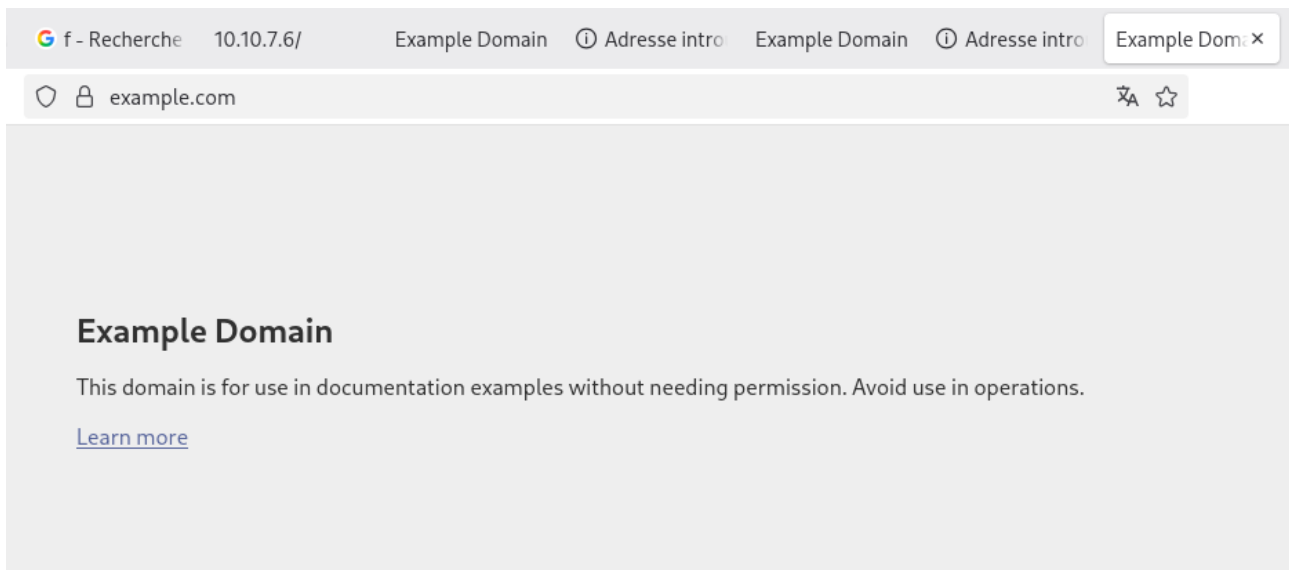
La connexion a échoué

Firefox ne peut établir de connexion avec le serveur à l'adresse example.com.

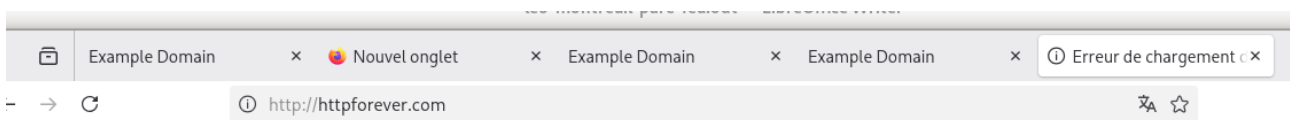
- Le site est peut-être temporairement indisponible ou surchargé. Réessayez plus tard ;
- Si vous n'arrivez à naviguer sur aucun site, vérifiez la connexion au réseau de votre ordinateur ;
- Si votre ordinateur ou votre réseau est protégé par un pare-feu ou un proxy, assurez-vous que Firefox est autorisé à accéder au Web.

Réessayer

Par contre , si l'on tente d'accéder au même site en https avec l'url « https://example.com », on peut voir que la connexion est autorisée et fonctionne :



un autre exemple avec le site « http://httpforever.com » qui fonctionne uniquement en http :



La connexion a échoué

Firefox ne peut établir de connexion avec le serveur à l'adresse httpforever.com.

- Le site est peut-être temporairement indisponible ou surchargé. Réessayez plus tard ;
- Si vous n'arrivez à naviguer sur aucun site, vérifiez la connexion au réseau de votre ordinateur ;
- Si votre ordinateur ou votre réseau est protégé par un pare-feu ou un proxy, assurez-vous que Firefox est autorisé à accéder au Web.

Réessayer

la connexion est bloqué aussi .

Le blocage HTTP est donc réussi, nous pouvons maintenant passer à l'hébergement de la page web qui elle sera la seule accessible en http .

4- Héberger une page accessible en http

Il faut tout d'abord créer la page, on peut faire cette commande pour créer une page simple pour le test :

```
root@debian:~# echo "<h1>Access redirige par iptables/léo</h1>" | sudo tee /var/www/html/index.html
<h1>Access redirige par iptables/léo</h1>
```

Nous avons déjà installé apache au début du tp qui sert à rendre accessible la page en local .

Maintenant il faut ajouter une règle qui autorise uniquement l'accès à cette page en http .

Il y a deux possibilités, soit on rend notre page web soit accessible par tout le monde en http y compris nous, soit on veut que notre page web ne soit accessible que par nous même en local et en http .

Pour la première possibilité, si on veut que tout le monde puisse accéder à la page web en http y compris nous , il suffit de rajouter une règle qui autorise tout le trafic entrant sur le port 80 . De ce fait, les autres machines du même réseau pourront accéder à la page en http car l'entrée est autorisée vers notre machine sur le port 80 .

Pour rajouter cette règle, on utilise cette commande :

```
root@debian:~# sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

Voici après cette commande un aperçu de toutes les règles de notre pare feu grâce à la commande « iptables -L »

```
root@debian:~# sudo iptables -L
Chain INPUT (policy DROP)
target     prot opt source                destination
ACCEPT     all  --  anywhere              anywhere
ACCEPT     all  --  anywhere              anywhere             ctstate RELATED,ESTABLISHED
ACCEPT     icmp --  anywhere              anywhere             icmp echo-request

Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
ACCEPT     all  --  anywhere              anywhere
REJECT     tcp  --  anywhere              anywhere             tcp dpt:http reject
t-with icmp-port-unreachable
```

Pour la deuxième possibilité, si l'on veut que notre page web ne soit accessible qu'en localhost, c'est à dire que par nous même en http, on rajoute une règle qui autorise le port 80 en entrée et en

sortie mais que sur le loopback c'est à dire localhost .

Pour rajouter la règle, on fait ces deux commandes :

-« sudo iptables -I OUTPUT -o lo -p tcp --dport 80 -j ACCEPT »

-« sudo iptables -I INPUT -i lo -p tcp --sport 80 -j ACCEPT »

De ce fait, la page web http ne sera pas accessible par les autres, elle sera que accessible via notre machine en localhost .

On peut d'ailleurs tester sur notre machine si la page web est accessible en http malgré le fait qu'on ai bloqué tout le http en sortie :



On peut voir que la page est bien accessible .

Il fallait aussi pour ce tp faire en sorte que notre page web au bout de 5 seconde redirige vers le site « http://example.com »

Pour ce faire, il suffit juste de modifier notre page web qui se trouve au chemin de base « var/www/html/index.html » et rajouter à l'intérieur la ligne : <meta http-equiv="refresh" content="5;url=http://example.com">.

Nous pouvons maintenant passer au blocage d'ip des voisins contigus .

5- Bloquer les IP des voisins contigus

Pour ce TP, mes voisins contigus avait pour ip 10.10.7.5 et 10.10.7.7 , il faut alors rajouter des règles dans le pare-feu qui vont bloqué ces IP de sorte à ce que quand elle vont nous ping, les paquets n'arriveront pas à nous atteindre .

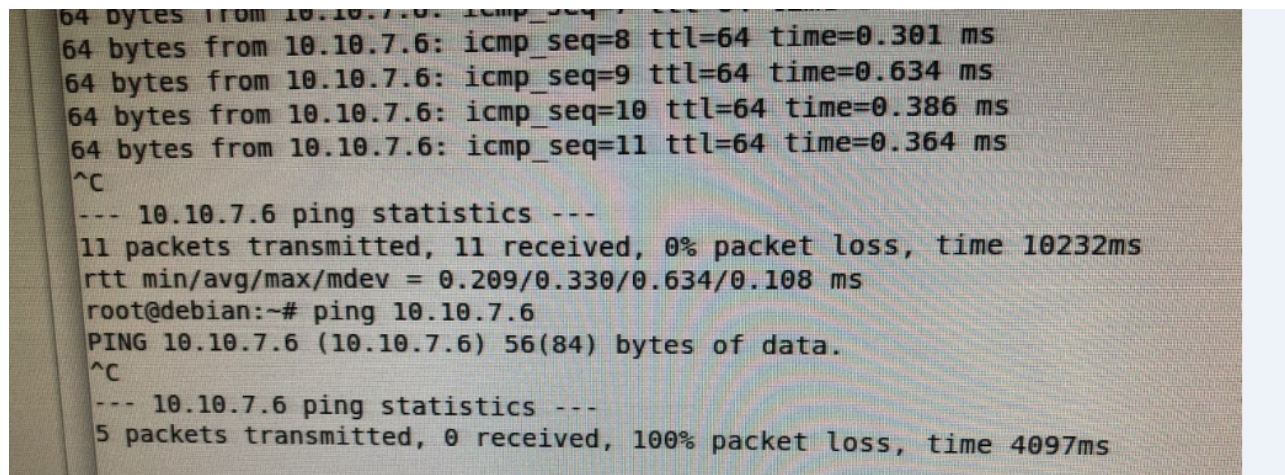
Pour ce faire, nous allons bloquer tout trafic venant de nos voisins contigus avec ces deux commandes :

```
root@debian:~# sudo iptables -I INPUT 1 -s 10.10.7.5 -j DROP
root@debian:~# sudo iptables -I FORWARD 1 -s 10.10.7.5 -j DROP
```

(Dans le cas ci dessus, j'ai bloqué l'ip 10.10.7.5, pour bloquer l'autre voisin, il suffit de remplacer l'ip par 10.10.7.7 mais je n'ai pas pris de screen quand je l'ai fait)

la première commande empêche tout le trafic du voisin d'arriver, la deuxième empêche le voisin de passer par moi pour aller vers d'autres réseau. De ce fait, les IP de mes voisins contigus sont complètement bloquer.

On peut d'ailleurs tester de ping notre machine via la machine d'un de nos voisins contigus et on peut voir que les paquets n'arrivent pas à destination et sont perdus :



On voit en haut du screen qu'avant les paquets arrivaient bien à destination mais après l'ajout des règles, les paquets sont perdus.

On a réussi à bloquer l'ip de nos voisins contigus, on peut maintenant passer au blocage du site « leboncoin »

6- Bloquer Leboncoin

Pour bloquer leboncoin il existe plusieurs méthodes. Pour ce TP, nous allons ajouter une règle qui bloque toutes les ip du site leboncoin de sorte à ne plus pouvoir accéder à leboncoin par n'importe quelle ip du site.

Pour ce faire, on va utiliser ces commandes :

```
root@debian:~# dig +short leboncoin.fr | while read ip; do
> sudo iptables -A OUTPUT -d "$ip" -j DROP
> done
```

C'est une boucle qui récupère toutes les adresses ip du site leboncoin et qui pour chaque ip du site leboncoin, applique la règle qui bloque le trafic sortant depuis ma machine vers l'ip.

En clair, ma machine ne peut plus envoyer la moindre requête, le moindre paquet vers n'importe quelle ip du site leboncoin.fr.

Il faut faire attention car le site change régulièrement d'ip, donc pour que le blocage continue de fonctionner tout le temps, il faut soit automatiser la boucle, soit manuellement la retaper car iptables ne met pas à jour les IP.

Pour tester si le blocage fonctionne il suffit de tenter d'accéder à leboncoin (j'ai oublié de prendre une capture d'écran mais ça fonctionne).

Source : Chatgpt